



ETHINX

Canonical Architecture

v1.0 Canonical 2026-06-08

Toggle Theme



ETHINX Canonical Architecture v1

Multi-tenant automation operating system for service businesses •
Source of Truth

DODO PAYMENTS SUPABASE

Version

v1.0

Date

2026-06-08

Status

Canonical Source of Truth

Owner

Platform Architecture

1. Vision & Product Boundaries

What ETHINX IS

A multi-tenant automation operating system for service businesses. Provides lead capture, enrichment, routing, workflow execution, and template-based verticalization—all on a single codebase with strict tenant isolation.

Not a CRM

No contact timeline builder, deal pipelines, or sales forecasting. Integrates with CRMs.

Not a Workflow Builder

No drag-and-drop DAG editor in v1. Workflows are template-defined, config-driven.

Not an Agency

Product-first. Templates are deployable artifacts, not services.

Core Thesis

One platform → many service industries → many templates → one codebase. Tenants get vertical-specific automation in minutes via template installation, without code forks or separate deployments.

Product Boundaries v1

- Lead Engine: capture, normalize, dedupe, score, assign
- Workflow Engine: trigger → action, synchronous + queued execution
- Template System: versioned bundles of workflows, forms, copy, UI config
- Multi-tenancy with RLS: absolute data isolation
- Billing via Dodo Payments only
- Admin, tenant management, audit logs

2. System Overview & Service Map

flowchart TB
U[User Browser] --> FE[Next.js 14 App Router Vercel]
FE --> SB[Supabase]
Postgres + Auth + RLS + Storage --> EF[Supabase Edge Functions]
EF --> WF[Workflow Engine]
EF --> DODO[Dodo Payments API]
EF --> LEAD[Lead Engine]
WF --> SB
LEAD --> SB
DODO --> WH[Webhook Handler]
WH --> SB
SB --> RLS[(Row Level Security)]
FE --> TMPL[Template System]
TMPL --> SB

Components

- **Next.js Frontend** – App Router, Server Actions, RSC
- **Supabase** – Postgres, Auth, RLS, Storage, Realtime, Edge Functions
- **Dodo Payments** – Checkout, subscriptions, webhooks, customer portal
- **Workflow Engine** – Trigger→Action runner with retry
- **Template System** – Installable vertical packs

Data Flow

1. User authenticates via Supabase Auth → tenant context set
2. Lead enters via form/API → Edge Function normalizes → RLS inserts
3. Workflow triggers on lead.created → actions execute
4. Checkout via Dodo → webhook provisions subscription → RLS grants features
5. Template install copies config + workflows into tenant scope

3. Multi-Tenancy Model

Tenant Isolation Strategy

Shared database, shared schema, row-level isolation via `tenant_id` on every tenant-scoped table. Enforced by Postgres RLS. No cross-tenant queries possible from application role.

RLS Approach

- auth.jwt() → app_metadata.tenant_id set on login
- All policies check tenant_id = current_tenant_id()
- Platform Admin bypass via service_role, audited
- Supabase Storage buckets namespaced by tenant_id

4. Database Schema v1

Core tables with RLS. Extensions: pgcrypto, uuid-oss.

tenants.sql

Copy

```
CREATE TABLE tenants (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  slug text UNIQUE NOT NULL,  
  name text NOT NULL,  
  dodo_customer_id text,  
  status text NOT NULL DEFAULT 'trialing' CHECK (status IN ('trialing','active')),  
  created_at timestamptz NOT NULL DEFAULT now(),  
  updated_at timestamptz NOT NULL DEFAULT now()  
);  
ALTER TABLE tenants ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "tenant_isolation_select" ON tenants FOR SELECT USING (tenant_id = current_tenant_id());  
CREATE POLICY "tenant_isolation_update" ON tenants FOR UPDATE USING (tenant_id = current_tenant_id());
```

users & memberships.sql

Copy

```
CREATE TABLE memberships (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  role text NOT NULL CHECK (role IN ('owner','admin','member','readonly')),  
  created_at timestamptz NOT NULL DEFAULT now(),  
  UNIQUE(tenant_id, user_id)
```

```
);
ALTER TABLE memberships ENABLE ROW LEVEL SECURITY;
CREATE POLICY "members_read_own_tenant" ON memberships FOR SELECT USING (
tenant_id = current_tenant_id() AND EXISTS (
  SELECT 1 FROM memberships m WHERE m.tenant_id = current_tenant_id()
  AND m.user_id = auth.uid() AND m.role IN ('owner','admin')
)
);
```

subscriptions & plans.sql

Copy

```
CREATE TABLE plans (
  id text PRIMARY KEY,
  name text NOT NULL,
  dodo_product_id text NOT NULL,
  dodo_price_id text NOT NULL,
  features jsonb NOT NULL DEFAULT '{}',
  limits jsonb NOT NULL DEFAULT '{}'
);

CREATE TABLE subscriptions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  dodo_subscription_id text UNIQUE NOT NULL,
  dodo_customer_id text NOT NULL,
  plan_id text NOT NULL REFERENCES plans(id),
  status text NOT NULL,
  current_period_end timestamptz,
  cancel_at_period_end boolean DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE subscriptions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "sub_isolation" ON subscriptions FOR SELECT USING (tenant_id = current_tenant_id());
```

leads.sql

Copy

```
CREATE TABLE leads (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  source text NOT NULL,  
  email text,  
  phone text,  
  name text,  
  payload jsonb NOT NULL DEFAULT '{}',  
  score integer DEFAULT 0,  
  status text NOT NULL DEFAULT 'new' CHECK (status IN ('new','working','closed')),  
  assigned_to uuid REFERENCES auth.users(id),  
  dedupe_key text,  
  created_at timestamptz NOT NULL DEFAULT now(),  
  updated_at timestamptz NOT NULL DEFAULT now()  
);  
CREATE INDEX ON leads(tenant_id, created_at DESC);  
CREATE INDEX ON leads(tenant_id, dedupe_key);  
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "leads_isolation" ON leads FOR ALL USING (tenant_id = current_user());
```

```
CREATE TABLE lead_events (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id uuid NOT NULL,  
  lead_id uuid NOT NULL REFERENCES leads(id) ON DELETE CASCADE,  
  type text NOT NULL,  
  data jsonb NOT NULL DEFAULT '{}',  
  created_at timestamptz NOT NULL DEFAULT now()  
);  
ALTER TABLE lead_events ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "lead_events_isolation" ON lead_events FOR ALL USING (tenant_id = current_user());
```

workflows.sql

Copy

```
CREATE TABLE workflows (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
```

```

key text NOT NULL,
name text NOT NULL,
trigger_type text NOT NULL,
trigger_config jsonb NOT NULL DEFAULT '{}',
actions jsonb NOT NULL DEFAULT '[]',
enabled boolean NOT NULL DEFAULT true,
created_at timestamptz NOT NULL DEFAULT now(),
UNIQUE(tenant_id, key)
);
ALTER TABLE workflows ENABLE ROW LEVEL SECURITY;
CREATE POLICY "wf_isolation" ON workflows FOR ALL USING (tenant_id =

CREATE TABLE workflow_runs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL,
  workflow_id uuid NOT NULL REFERENCES workflows(id) ON DELETE CA
  trigger_event jsonb NOT NULL,
  status text NOT NULL DEFAULT 'running',
  output jsonb,
  error text,
  started_at timestamptz NOT NULL DEFAULT now(),
  finished_at timestamptz
);
ALTER TABLE workflow_runs ENABLE ROW LEVEL SECURITY;
CREATE POLICY "wf_runs_isolation" ON workflow_runs FOR ALL USING (te

```

templates & audit.sql

Copy

```

CREATE TABLE templates (
  id text PRIMARY KEY,
  name text NOT NULL,
  industry text NOT NULL,
  version text NOT NULL,
  manifest jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE template_installations (

```

```
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id uuid NOT NULL REFERENCES tenants(id) ON DELETE CASCADE
template_id text NOT NULL REFERENCES templates(id),
version text NOT NULL,
config jsonb NOT NULL DEFAULT '{}',
installed_at timestamptz NOT NULL DEFAULT now(),
UNIQUE(tenant_id, template_id)
);
ALTER TABLE template_installations ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tpl_inst_isolation" ON template_installations FOR ALL U
```

```
CREATE TABLE audit_logs (
  id bigserial PRIMARY KEY,
  tenant_id uuid,
  actor_id uuid,
  action text NOT NULL,
  entity text NOT NULL,
  entity_id text,
  diff jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY "audit_tenant_read" ON audit_logs FOR SELECT USING (ten
```

5. User Roles & Permissions

Role	Manage Billing	Manage Members	Edit Workflows	View Leads	Edit Leads	Install Templates
Platform Admin	✓	✓	✓	✓	✓	✓
Tenant Owner	✓	✓	✓	✓	✓	✓
Tenant Admin	✗	✓	✓	✓	✓	✓
Tenant User	✗	✗	✗	✓	✓	✗
Read-only	✗	✗	✗	✓	✗	✗

6. Subscription & Billing with Dodo

Dodo is the sole payment processor

No Stripe. All checkout sessions, subscriptions, invoices, and customer portal are powered by Dodo Payments API.

Plans Structure

Plan ID	Name	Dodo Product	Monthly	Limits
starter	Starter	prod_eth_starter	\$49	1k leads/mo, 3 workflows
growth	Growth	prod_eth_growth	\$149	10k leads/mo, unlimited workflows
scale	Scale	prod_eth_scale	\$399	50k leads/mo, priority support

Dodo Integration Flow

sequenceDiagram participant User participant FE as Next.js participant EF as Edge Function participant DODO as Dodo API participant DB as Supabase User->>FE: Click Upgrade FE->>EF: create-checkout-session(plan_id) EF->>DODO: POST /checkout/sessions DODO-->>EF: checkout_url EF-->>FE: redirect_url User->>DODO: Complete payment DODO->>EF: webhook payment.succeeded EF->>DB: upsert subscription + tenant.status = active DODO->>EF: subscription.created EF->>DB: provision limits

Webhook Handlers

```
// /api/webhooks/dodo
export async function POST(req) {
  const sig = req.headers.get('dodo-signature');
  const raw = await req.text();
  const event = verifyDodoWebhook(raw, sig); // HMAC
```

```

switch(event.type) {
  case 'payment.succeeded':
    await provisionTenant(event.data.customer_id, event.data.subscription_id);
    break;
  case 'subscription.created':
  case 'subscription.updated':
    await upsertSubscription(event.data);
    break;
  case 'subscription.cancelled':
    await markPastDue(event.data.subscription_id);
    break;
}
return Response.json({ received: true });
}

```

7. Revenue Model

- **SaaS Subscription Tiers** – Starter / Growth / Scale via Dodo
- **Template Marketplace (Phase 2)** – 70/30 revenue share with template creators, payouts via Dodo Connect
- **Add-ons** – Extra lead volume, dedicated webhook endpoints

8. Workflow Engine Design v1

Simple trigger → action model. No DAG. Runs in Edge Function with queue fallback.

```

{
  "key": "lead_new_notify",
  "trigger_type": "lead.created",
  "trigger_config": { "min_score": 50 },
  "actions": [
    { "type": "send_email", "config": { "template": "lead_alert" } },
    { "type": "webhook", "config": { "url": "{{tenant.webhook_url}}" } },
    { "type": "assign_user", "config": { "round_robin": true } }
  ]
}

```

Supported Triggers v1

lead.created, lead.updated, subscription.created, form.submitted

Supported Actions v1

send_email, send_sms, webhook, assign_user, update_lead, create_task

9. Template System

Templates bundle workflows, forms, copy, and UI config for a vertical.

Home Services

Emergency call routing, quote request form, SMS follow-up, Google Review request workflow.

Health & Wellness

HIPAA-safe intake, appointment reminder, no-show reactivation, consent capture.

Professional Services

Consult booking, qualification form, proposal send, nurture sequence.

Installation Flow

1. User selects template → preview manifest
2. Edge Function clones workflows with tenant_id
3. Forms and copy written to tenant config
4. template_installations record created

10. Lead Engine v1 Specification

Inputs

Web forms, Facebook Lead Ads, Email parsing, Phone (Twilio webhook)

Process

Normalize → Deduplicate → Score → Assign

Outputs

CRM record, Notification, Workflow trigger

API Endpoints

POST /api/leads/ingest # public, HMAC signed

GET /api/leads

PATCH /api/leads/:id

POST /api/leads/:id/events

Acceptance Criteria

- Dedupe within 5 min window by email + phone hash
- Score 0-100 based on source + fields completeness
- Assignment respects user capacity and round-robin
- All writes audit logged

11. Deployment Architecture

- **Supabase Project** – Production + Staging
- **Vercel** – Next.js with Edge Runtime
- **Edge Functions** – Deno, deployed via Supabase CLI
- **Env Vars** – DODO_API_KEY, DODO_WEBHOOK_SECRET, SUPABASE_URL, SUPABASE_SERVICE_ROLE
- **CI/CD** – GitHub Actions: test → migrate → deploy

12. API Contracts

openapi: 3.0.3

info:

title: ETHINX API

```
version: 1.0.0
paths:
  /api/leads:
    get:
      summary: List leads
      parameters:
        - name: status
          in: query
          schema: { type: string }
    post:
      summary: Create lead
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                email: { type: string }
                phone: { type: string }
                source: { type: string }
  /api/workflows:
    get: { summary: List workflows }
  /api/tenants/me:
    get: { summary: Current tenant }
```

13. Manus-Optimised Build Instructions Template

```
### TASK: Implement Dodo Checkout Session
**Requirements**
- Create Edge Function `create-checkout-session`
- Input: plan_id, tenant_id
- Call Dodo API with customer email
- Return checkout_url

**Tables**
- tenants (dodo_customer_id)
- plans (dodo_price_id)
```

****Success Criteria****

- 200 response with url
- Webhook `payment.succeeded` provisions subscription
- RLS prevents cross-tenant access
- Tests pass

14. Roadmap Phases

Phase 1: Revenue Engine

Dodo billing, lead engine, 3 templates, workflow v1

Phase 2: Template Marketplace

Creator portal, versioning, revenue share

Phase 3: Operator Ecosystem

Partner onboarding, white-label

Phase 4: AI Optimisation

Lead scoring ML, workflow suggestions

Phase 5: Autonomous Deployment

Self-provisioning tenants, auto-template fitting

15. Deliberately Out of Scope for v1

- Agent choreography / multi-agent orchestration
- DAG workflow editor
- Multiple Cloud Run services / microservices
- Complex event orchestration / Saga patterns
- Custom code execution in workflows
- Bring-your-own database
- Stripe, Paddle, or other payment processors

ETHINX Canonical Architecture v1 • © 2026 ETHINX Labs. This document is the Source of Truth for Manus builds. Payment

processing is exclusively via Dodo Payments.